

Report 1 — Reservoir Model for OpenHPL/OpenHPLjl

Madhusudhan Pandey

September 20, 2026

Abstract

This report defines the reservoir as the first physical component in an equation-based hydropower modeling workflow using ModelingToolkit.jl and the SciML ecosystem. It follows the OpenHPLjl documentation sequence: updated context, generalized concept, literature model classes, OpenHPL mathematics and interpretation, Julia implementation, assembly, validation, expected outputs, and engineering interpretation.

Contents

1	Project Development Flow	3
2	Updated Context	3
3	Generalized Concept Model	3
4	Other Models Available in the Literature	4
4.1	Infinite or Constant-Head Reservoir	4
4.2	Lumped Constant-Area Reservoir	4
4.3	Nonlinear Geometry Reservoir	4
4.4	Reservoir With Hydrological Dynamics	4
4.5	Distributed or Spatial Reservoir Models	4
5	Mathematics Involved in the OpenHPL Concept/Model	5
5.1	Fundamental Conservation Law	5
5.2	Constant-Area Reservoir	5
5.3	Nonlinear Reservoir Geometry	5
5.4	Hydraulic Pressure and Head	5
5.5	State-Space / DAE Form	5
6	OpenHPL-Specific Modeling Interpretation	6
7	Implementation in Julia ModelingToolkit / SciML	6
7.1	Hydraulic Connector	6
7.2	Infinite Reservoir Model	6
7.3	Constant-Area Dynamic Reservoir	6
7.4	Nonlinear Geometry Reservoir	7
8	Model Assembly and Connections	7
9	Numerical Experiment / Validation	8
10	Expected Outputs	8

11 Engineering Interpretation	9
Conclusion	9

1 Project Development Flow

The fresh OpenHPLjl kickoff established a minimal ModelingToolkit/SciML package and removed older experimental scaffolding from the active tree. The following commits added this reservoir report and linked the technical-report workflow from the project README.

This branch, `reservoir-model`, is the first component-development branch. Its purpose is to move from documented mathematics to tested equation-based implementation:

Report 1 → HydraulicPort → InfiniteReservoir → DynamicReservoir → mass-balance tests → validation

The next implementation commit should therefore introduce the hydraulic connector, constant-head and finite-storage reservoir components, analytical conservation tests, and one minimal numerical experiment. Once this milestone is validated, the next component will be the rigid waterway model based on momentum conservation and water inertia.

2 Updated Context

A reservoir is normally the upstream hydraulic boundary of a hydropower plant. It stores gravitational potential energy and establishes the hydraulic head available to the downstream waterway.

Reservoir → Intake → Waterway → Surge Tank → Penstock → Turbine → Shaft → Generator → Grid

OpenHPL is an open-source Modelica hydropower library containing waterway, electromechanical, controller, interface, and example models. The current OpenModelica-generated documentation identifies version 3.0.1 [1]. Its user guide states that reservoirs, conduits, surge tanks, and fittings are based on mass and momentum balances, and that a hydrology model is available for reservoir inflow studies [2].

For short dynamic studies such as governor response or frequency containment reserve (FCR), the upper reservoir can often be approximated as a constant-head boundary. For longer simulations, reservoir storage dynamics should be retained.

The objective of the OpenHPLjl reservoir component is to provide a reusable equation-based model that can range from a constant-head reservoir to a nonlinear finite-storage reservoir without changing the downstream hydraulic interface.

3 Generalized Concept Model

The most general lumped reservoir representation is

$$\boxed{\text{Inflow} \rightarrow \text{Storage} \rightarrow \text{Outflow}}, \quad Q_{\text{in}} \rightarrow V(H) \rightarrow Q_{\text{out}}.$$

A generalized hydraulic connector contains

$$\boxed{H, \quad Q},$$

where H is an across/potential variable and Q is a through/flow variable. For two connected components,

$$H_1 = H_2, \quad Q_1 + Q_2 = 0.$$

Table 1: Principal reservoir variables.

Quantity	Symbol	SI unit
Reservoir volume	V	m^3
Water level / hydraulic head	H	m
Inflow	Q_{in}	$\text{m}^3 \text{s}^{-1}$
Outflow	Q_{out}	$\text{m}^3 \text{s}^{-1}$
Free-surface area	A	m^2
Water density	ρ	kg m^{-3}
Gravitational acceleration	g	m s^{-2}

4 Other Models Available in the Literature

4.1 Infinite or Constant-Head Reservoir

$$H(t) = H_0.$$

There is no dynamic reservoir state. This is useful for short studies dominated by turbine, governor, waterway, or grid dynamics.

4.2 Lumped Constant-Area Reservoir

For a finite reservoir with approximately constant area,

$$V = A_r H, \quad A_r \frac{dH}{dt} = Q_{\text{in}} - Q_{\text{out}}.$$

4.3 Nonlinear Geometry Reservoir

For realistic geometry,

$$V = V(H), \quad A(H) = \frac{dV}{dH},$$

so

$$A(H)\dot{H} = Q_{\text{in}} - Q_{\text{out}}.$$

The function $V(H)$ can come from surveyed geometry, fitted polynomials, or lookup tables.

4.4 Reservoir With Hydrological Dynamics

A longer-term model can use

$$Q_{\text{in}} = Q_{\text{rain}} + Q_{\text{snow}} + Q_{\text{catchment}} - Q_{\text{evaporation}}.$$

OpenHPL includes hydrological modeling capability for reservoir inflow [2].

4.5 Distributed or Spatial Reservoir Models

For large lakes or flood studies, water level may vary spatially and PDE-based shallow-water or free-surface equations may be required.

A useful hierarchy is

Constant head $\rightarrow$ Lumped storage $\rightarrow$ Nonlinear geometry $\rightarrow$ Hydrological reservoir $\rightarrow$ Distributed PDE model

5 Mathematics Involved in the OpenHPL Concept/Model

5.1 Fundamental Conservation Law

For incompressible water,

$$\frac{dm}{dt} = \dot{m}_{\text{in}} - \dot{m}_{\text{out}}, \quad m = \rho V.$$

With approximately constant density,

$$\boxed{\frac{dV}{dt} = Q_{\text{in}} - Q_{\text{out}}}.$$

5.2 Constant-Area Reservoir

If $V = A_r H$,

$$\boxed{A_r \dot{H} = Q_{\text{in}} - Q_{\text{out}}}, \quad \boxed{\dot{H} = \frac{Q_{\text{in}} - Q_{\text{out}}}{A_r}}.$$

The dynamic state is $x = H$.

5.3 Nonlinear Reservoir Geometry

For $V = V(H)$,

$$\dot{V} = \frac{dV}{dH} \dot{H}.$$

With $A(H) = dV/dH$,

$$\boxed{A(H) \dot{H} = Q_{\text{in}} - Q_{\text{out}}}.$$

For example,

$$A(H) = a_0 + a_1 H + a_2 H^2$$

gives

$$\dot{H} = \frac{Q_{\text{in}} - Q_{\text{out}}}{a_0 + a_1 H + a_2 H^2}.$$

5.4 Hydraulic Pressure and Head

At an outlet elevation z_o ,

$$p_o = p_{\text{atm}} + \rho g(H - z_o).$$

Hydraulic head can be written

$$H_p = \frac{p}{\rho g} + z.$$

5.5 State-Space / DAE Form

A convenient organization is

$$x = H, \quad z = \begin{bmatrix} V \\ A \end{bmatrix}, \quad u = \begin{bmatrix} Q_{\text{in}} \\ Q_{\text{out}} \end{bmatrix}.$$

The equation-based form is

$$\dot{x} = f(x, z, u, p), \quad 0 = g(x, z, u, p),$$

with

$$\dot{H} = \frac{Q_{\text{in}} - Q_{\text{out}}}{A(H)}, \quad V = V(H), \quad A = \frac{dV}{dH}.$$

6 OpenHPL-Specific Modeling Interpretation

OpenHPL is a component-oriented Modelica library whose waterway models are constructed from conservation equations and connected through physical interfaces [2]. The published Trollheim case study used OpenHPL to model, tune, and validate a real hydropower plant against experimental measurements [3].

Conceptually,

$$\boxed{\text{Reservoir.port} \longleftrightarrow \text{Waterway.port}}.$$

For a fixed reservoir,

$$H_{\text{port}} = H_0.$$

For a dynamic reservoir, when port flow is positive into the component,

$$H_{\text{port}} = H(t), \quad A(H)\dot{H} = Q_{\text{in}} + Q_{\text{port}}.$$

During normal generation at an outlet port, $Q_{\text{port}} < 0$.

7 Implementation in Julia ModelingToolkit / SciML

ModelingToolkit supports acausal connection semantics. Variables marked with `connect = Flow` sum to zero over a connection set, while ordinary connector variables are equated [4].

7.1 Hydraulic Connector

```
using ModelingToolkit
using ModelingToolkit: t_nounits as t, D_nounits as D

@connector HydraulicPort begin
    H(t), [description = "Hydraulic head [m]"]
    Q(t), [connect = Flow,
           description = "Volumetric flow into component [m^3/s]"]
end
```

7.2 Infinite Reservoir Model

```
@mtkmodel InfiniteReservoir begin
    @parameters begin
        H0 = 100.0,
        [description = "Constant reservoir head [m]"]
    end
    @components begin
        port = HydraulicPort()
    end
    @equations begin
        port.H ~ H0
    end
end
```

7.3 Constant-Area Dynamic Reservoir

```

@mtkmodel Reservoir begin
  @parameters begin
    A = 1.0e6,
    [description = "Free-surface area [m^2]"]
    H0 = 100.0,
    [description = "Initial water level [m]"]
  end
  @variables begin
    H(t) = H0,
    [description = "Reservoir water level [m]"]
    Qin(t) = 0.0,
    [description = "External inflow [m^3/s]"]
  end
  @components begin
    port = HydraulicPort()
  end
  @equations begin
    A * D(H) ~ Qin + port.Q
    port.H ~ H
  end
end

```

7.4 Nonlinear Geometry Reservoir

```

@mtkmodel NonlinearReservoir begin
  @parameters begin
    a0 = 1.0e6
    a1 = 0.0
    a2 = 0.0
    H0 = 100.0
  end
  @variables begin
    H(t) = H0
    Qin(t) = 0.0
    Aeff(t)
  end
  @components begin
    port = HydraulicPort()
  end
  @equations begin
    Aeff ~ a0 + a1*H + a2*H^2
    Aeff * D(H) ~ Qin + port.Q
    port.H ~ H
  end
end

```

8 Model Assembly and Connections

```

@named reservoir = Reservoir()
@named pipe = RigidPipe()

connections = [
  connect(reservoir.port, pipe.inlet)
]

```

The connection contributes

$$H_{\text{reservoir}} = H_{\text{pipe,in}}, \quad Q_{\text{reservoir}} + Q_{\text{pipe,in}} = 0.$$

The future chain can evolve as

$$\boxed{\text{Reservoir} \rightarrow \text{RigidPipe} \rightarrow \text{SurgeTank} \rightarrow \text{Penstock} \rightarrow \text{Turbine}}.$$

9 Numerical Experiment / Validation

Consider

$$A = 1000 \text{ m}^2, \quad Q_{\text{in}} = 10 \text{ m}^3 \text{ s}^{-1}, \quad Q_{\text{out}} = 12 \text{ m}^3 \text{ s}^{-1}.$$

Then

$$\dot{H} = \frac{10 - 12}{1000} = -0.002 \text{ m s}^{-1}.$$

After 10 s,

$$H(10) - H(0) = -0.02 \text{ m},$$

hence

$$\boxed{H(10) = H_0 - 0.02 \text{ m}}.$$

```
using ModelingToolkit
using OrdinaryDiffEq

compiled = mtkcompile(model)
prob = ODEProblem(compiled, [], (0.0, 10.0))
sol = solve(prob, Tsit5())
```

For larger coupled hydraulic systems, solver selection should be revisited based on stiffness, algebraic structure, and initialization requirements.

10 Expected Outputs

For a constant-head reservoir,

$$H(t) = H_0.$$

For a dynamic reservoir, useful outputs are

$$H(t), \quad V(t), \quad Q_{\text{in}}(t), \quad Q_{\text{out}}(t),$$

and, where useful,

$$P_h(t) = \rho g Q H.$$

Recommended plots are reservoir level, inflow/outflow, stored volume, and a mass-balance residual. For the explicit outflow convention,

$$r_m = \dot{V} - Q_{\text{in}} + Q_{\text{out}}, \quad r_m \approx 0.$$

For the connector convention,

$$\dot{V} = Q_{\text{in}} + Q_{\text{port}},$$

and the residual must use the same sign convention.

11 Engineering Interpretation

The reservoir establishes a core OpenHPLjl principle:

Do not prescribe signal direction unnecessarily.

Instead, define physical variables, connector semantics, and conservation laws.

For early SMIB and FCR studies, a useful starting point is

$$H = H_0,$$

while longer studies can use

$$A(H)\dot{H} = Q_{\text{in}} - Q_{\text{out}}.$$

Thus,

InfiniteReservoir $\subset$ DynamicReservoir $\subset$ NonlinearReservoir

can preserve one hydraulic connection concept while increasing model fidelity.

Conclusion

The reservoir is the natural first component because it introduces

physical conservation + acausal connectors + DAE formulation + component reuse.

Its fundamental law is

$$\frac{dV}{dt} = Q_{\text{in}} - Q_{\text{out}},$$

or

$$A(H)\dot{H} = Q_{\text{in}} - Q_{\text{out}}.$$

The next component is the rigid waterway/rigid pipe, which introduces momentum balance and water inertia.

References

- [1] OpenHPL documentation, *Open-source hydropower library, Version 3.0.1*. <https://build.openmodelica.org/Documentation/OpenHPL.html>
- [2] OpenHPL, *User's Guide*. <https://build.openmodelica.org/Documentation/OpenHPL.UsersGuide.html>
- [3] L. Vytvytskyi and B. Lie, OpenHPL for Modelling the Trollheim Hydropower Plant, *Energies*, 12(12), 2303, 2019. <https://doi.org/10.3390/en12122303>
- [4] SciML, *ModelingToolkit.jl: Acausal Component-Based Modeling*. https://docs.sciml.ai/ModelingToolkit/stable/tutorials/acausal_components/